



Shiju Varghese's Masterclass

Professional Go

<https://shijuvar.medium.com>

About Me

=> 2.5 decades of experience in backend programming and distributed systems

- Senior Solutions Architect focused on Distributed Systems Architectures
- Providing Consulting and Training on Go and Distributed Systems
- Published Author: *Web Development with Go* (Apress, 2015, US edition), *Go Recipes* (Apress, 2016, US edition)
- Awarded Microsoft MVP seven times
- Speaker at numerous conferences
- medium.com/@shijuvar | github.com/shijuvar | linkedin.com/in/shijuvar



Go

- Go is an open source programming language created at Google in 2007 by Robert Griesemer, Rob Pike, and Ken Thompson
- First appeared in November, 2009
- Go 1.0 was released on March, 2012



Go is an open source programming language that enables the the production of **simple**, **efficient** and **reliable** software at scale.

The key point here is our programmers are Googlers, they're not researchers. They're typically, fairly young, fresh out of school, probably learned Java, maybe learned C or C++, probably learned Python. They're not capable of understanding a brilliant language but we want to use them to build good software. So, the language that we give them has to be easy for them to understand and easy to adopt.

– Rob Pike, Co-Designer, Go Programming Language

Go's largest goal is scale

- Creating software at scale
- Running software at scale

Introduction to Go

- An open source programming language created at Google
- Go is simple, minimal and pragmatic
- Modern general-purpose language
- A statically typed language with high productivity
- Compiles to native machine code; Compiles programs quickly
- Garbage-collected language
- Concurrency is a built-in feature

Go Versions

- First appeared in November, 2009
- Go 1.0 was released in March, 2012
- Current version is 1.23 (Released in August 2024)

Unlearning is the most important thing to master on
Go language

Gopher: An Iconic Mascot of Go



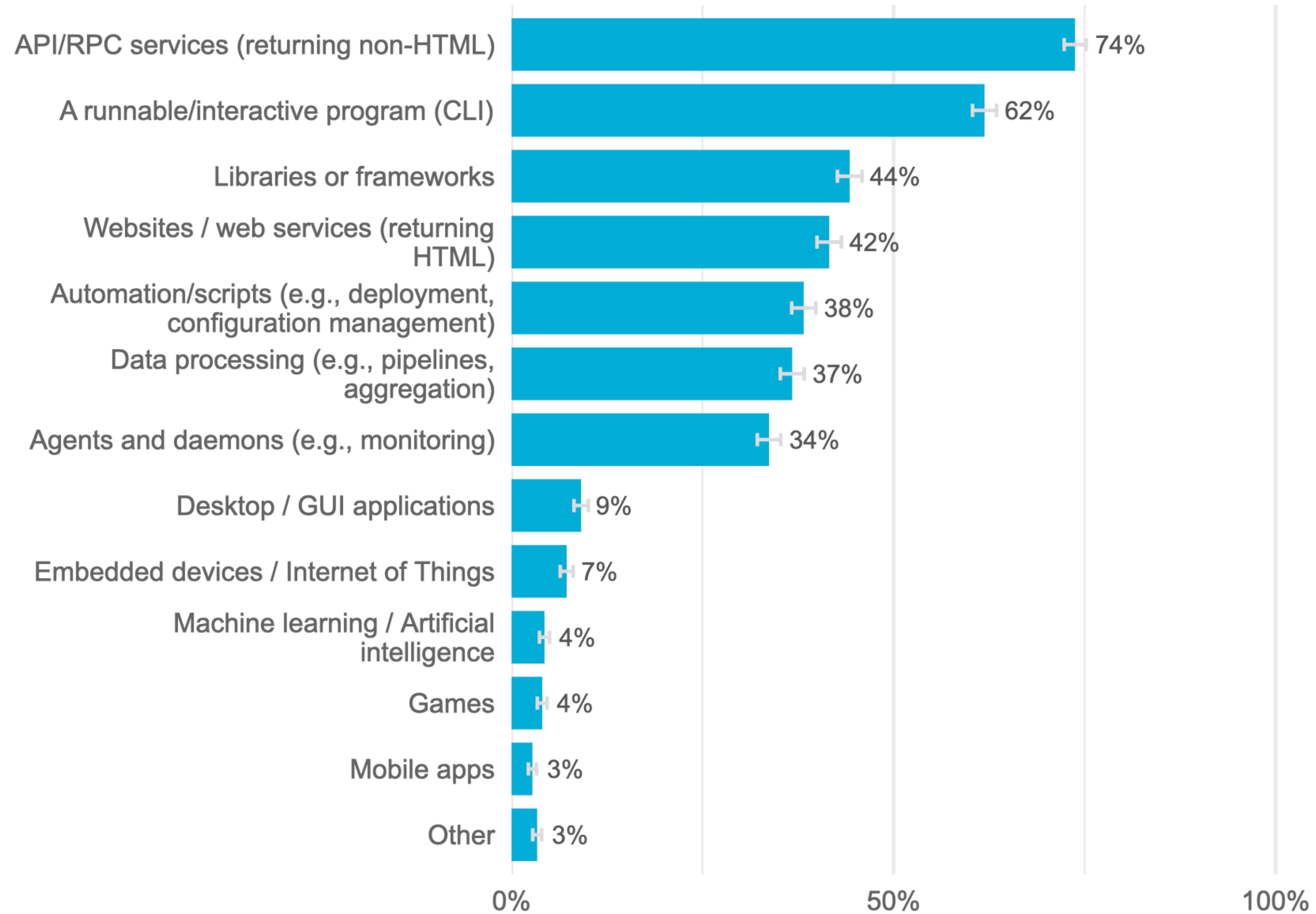
<https://blog.golang.org/gopher>

Why Go

- Built for writing high performance concurrent programming.
- Modern C for 21st Century.
- Pragmatism and Developer productivity.
- Easier Adoption.
- Go Community.
- Go success stories for building massively scalable applications.
- Language for the era of Cloud, Microservices and Containers.

What types of things do you build with Go?

(select all that apply)



% of respondents

<https://go.dev/solutions#case-studies>

Companies using Go

Organizations in every industry use Go to power their software and services [View all stories](#)



FACEBOOK



NETFLIX

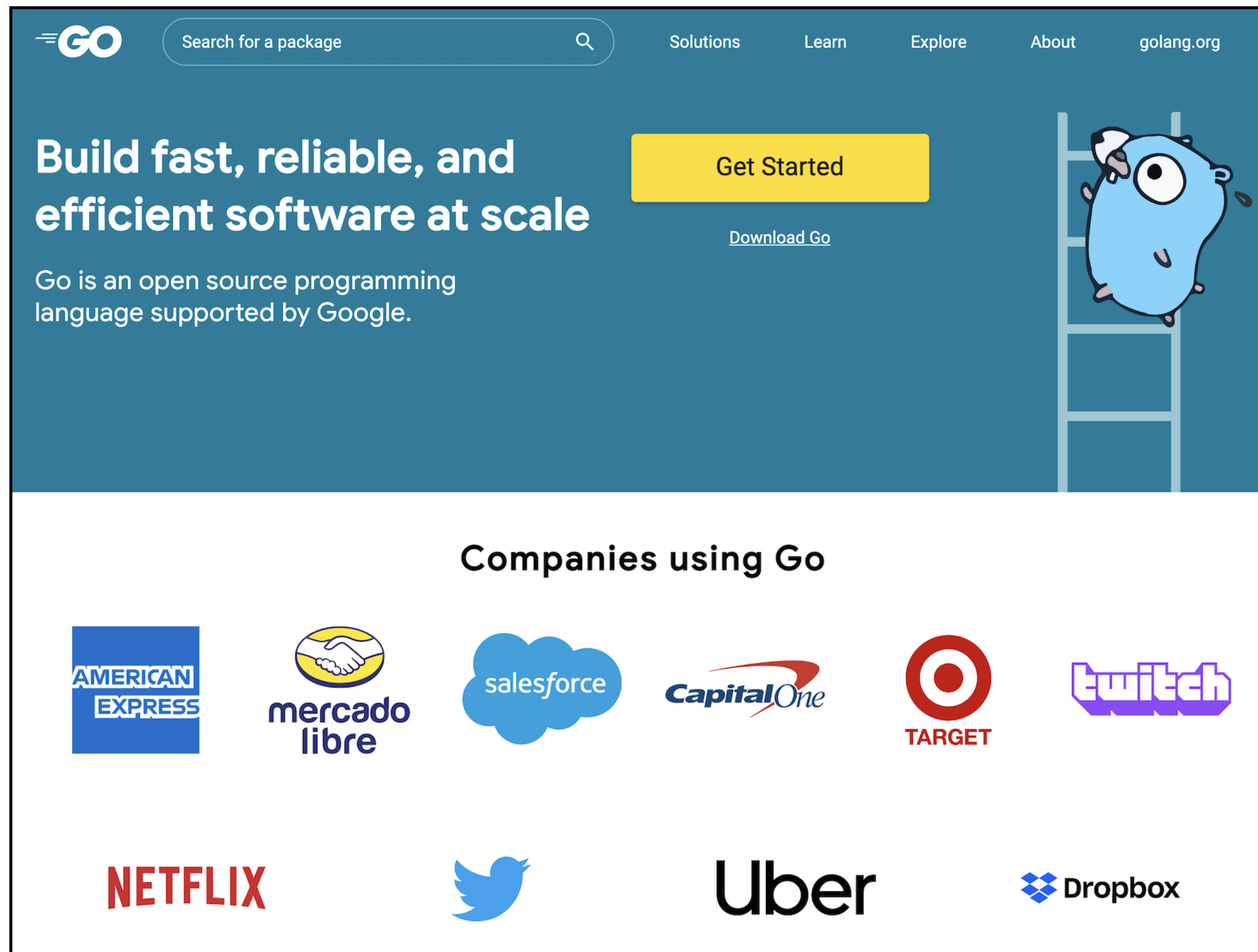


twitch



Uber

<https://go.dev>: A hub for Go developers



[Solutions](#)[Learn](#)[Explore](#)[About](#)[golang.org](#)

In-person training

[Ardan Labs](#)

Offering customized on-site live training classes.

[Gopher Guides](#)

Customized In-person, remote, and online training classes. Training for Developers by Developers.

[Boss Sauce Creative](#)

Personalized or track-based Go training for teams.

[Shiju Varghese](#)

On-site classroom training on Go and consulting on distributed systems architectures, in India.

Hands-on training for writing idiomatic Go code
with Clean architecture and SOLID principles

Source Code: <https://github.com/shjuvar/gokit> (Go basics to
architecture references)

Go Ecosystem

- Language
- Tools
- Packages

Setting up Go development environment

Set up Go development environment

- Download and install **Go**: <https://go.dev>
- Set the **GOMODCACHE** environment variable (Eg: export GOMODCACHE=\$HOME/gocode/pkg). The directory where third-party libraries will be downloaded
- Set the **GOBIN** environment variable - The directory where go commands will place binaries after building *main* packages. It's better set to a directory on the system PATH so that installed binaries can be run and discovered easily.
- Install **GIT**: <https://git-scm.com>
- Install an **IDE**, VS Code: <https://code.visualstudio.com> GoLand: <https://www.jetbrains.com/go/>

GOMODCACHE environment variable

- The module cache is the directory where the go command stores downloaded module files. The module cache is distinct from the build cache, which contains compiled packages and other build artifacts.

Variable Declarations

- The keyword `var` is used to declare variables.
- Inside the functions, short assignment operator (`:=`) can be used for declare and initialise variable without using `var` and datatypes.
- Short assignment operator (`:=`) can't be used for declare package level variables.
- The keyword `const` is used to declare constant variables.

Functions

- The keyword `func` is used for declare functions.
- Functions can return multiple values.
- Functions can have variadic parameter by using `...` syntax (eg: `...int`).
The variadic parameter should be the last parameter.
- Go supports anonymous functions.

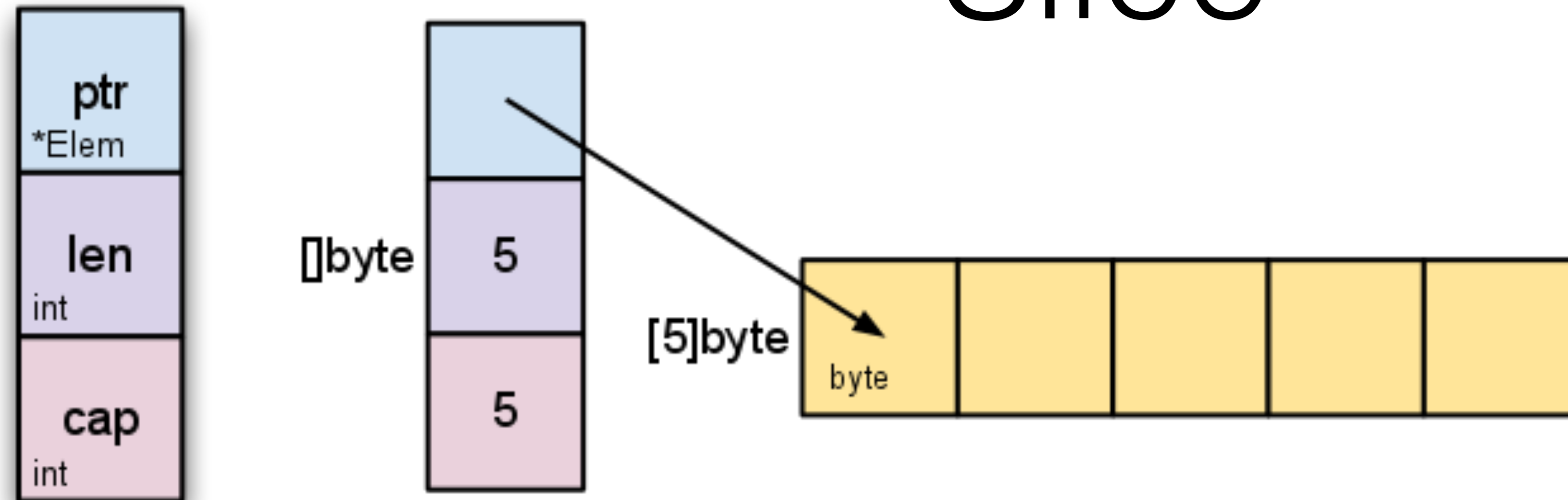
Functional Go

- Functional programming is based on immutable data structures: No mutable data; No state (or implicit state)
- Functions are first-class citizen in the Go programming language.
- Go functions are like values.
- Go supports Higher-order functions:
 - Takes one or more functions as parameters.
 - Functions can return functions.
- Go supports Closure - A function value that references variables declared outside its body.

Collections

- Go provides three types of data structure to works collections: **array**, **slice** and **map**
- Arrays are fixed length collection.
- Slices are dynamically sized collections.
- Maps are collection of key/value pairs.
- Slice and Map are reference types, which must be initialised either by using built-in **make** function or literal expression.

Slice



- Slice is a three-field data structure: a pointer points an underlying array, length specifies the length of an underlying array, and capacity specifies the capacity of the array.
- Slices can be enlarged at any time using built-in function **append**.
- A built-in function `copy` can be used for creating bigger slices by copying contents from an existing slice.

Map

- Go provides a built-in map type that implements a hash table.
- Map is a collection of key/value pairs, which is an unordered collection.
- The built-in **delete** function removes an entry from the map for the given key.
- Map provides a faster data retrieval for the given key.
- To iterate over the contents of a map, use the range keyword.

Defer, Panic, Recover

- The keyword defer is used to schedule a function call to be executed right after a function returns.
- The built-in panic function is used to stop the execution that will provide a panic situation.
- The built-in recover function is called within a deferred function to regain control over a panicking function.

Packages

- Go programs are organized into directories called packages.
- Go provides two kinds of packages: package main for which the resulting binary will be an executable, and shared library package to be used for other packages..
- Package main should have an entry point: main function.
- Packages identifiers started with uppercase letter will be exported to other packages.
- Packages identifiers started with lowercase letter will not be exported to other packages, but it will be available across the same package.

- Packages can have an init function, which can be used for writing initialising logic at packages level. This will be invoked before invoking main function.
- The keyword import is used to import packages into Go source files.
- Packages can be imported by providing an alias name.
- Packages can be imported by providing _ (underscore) as an alias name, which is used for importing a package just for invoking init functions.
- Third-party packages are installed using go get command.
- The ***Go Modules*** introduced in Go 1.11 as an experimental feature, has become an official solution for dependency management since Go 1.16

Go Modules

Challenges with Package Dependencies

- Dependency management of third-party packages
- go get always take latest version of packages

Go Modules

- Modules are how Go manages dependencies
- A module is a collection of related Go packages that are versioned together as a single unit
- A module is defined by a tree of Go source files with a **go.mod** file in the tree's root directory.
- Modules record precise dependency requirements and create reproducible builds
- Modules must be semantically versioned in the form v(major).(minor).(patch), such as v0.1.0, v1.2.3, or v3.0.1 (Check out: <https://semver.org>)
- Uses minimal version selection (MVS) algorithm

Workflow with Go Modules

- **'go mod init'** initialises a module with a file **go.mod**
- 'go get' will automatically update the go.mod file.
- 'go get' allows version selection to be overridden by adding an @version suffix or "module query" to the package argument
 - go get github.com/gorilla/mux@v1.6.2
 - go get github.com/gorilla/mux@'<v1.6.2'

Go Module Commands

- **go mod tidy** - go mod tidy ensures that the go.mod file matches the source code in the module. It adds any missing module requirements necessary to build the current module's packages and dependencies, and it removes requirements on modules that don't provide any relevant packages. It also adds any missing entries to go.sum and removes unnecessary entries.
- **go mod edit** - Provides a command-line interface for editing and formatting go.mod files, for use primarily by tools and scripts. (Eg: go mod edit -replace mycopmany.io/greetings=../greetings)
- **go mod download** - Downloads the named modules into the module cache (GOMODCACHE)
- **go list -m** - Print path of main module
- **go list -m -f={{.Dir}}** - Print root directory of main module
- **go list -m all** - View final versions that will be used in a build for all direct and indirect dependencies
- **go get -u ./...** or **go get -u=patch ./...** (from module root directory) - Update all direct and indirect dependencies to latest minor or patch upgrades
- **go build ./...** or **go test ./...** (from module root directory) - Build or test all packages in the module

Minimal Version Selection (MVS)

- The minimal version selection algorithm is used to select the versions of all modules used in a build. For each module in a build, the version selected by minimal version selection is always the semantically highest of the versions explicitly listed by a require directive in the main module or one of its dependencies.
- **MVS Selection:** If your module depends on module A which has a require D v1.0.0, and your module also depends on module B which has a require D v1.1.1, then minimal version selection would choose v1.1.1 of D to include in the build. This selection of v1.1.1 remains consistent even if sometime later a v1.2.0 of D becomes available.

GOMODCACHE environment variable

- The module cache is the directory where the go command stores downloaded module files. The module cache is distinct from the build cache, which contains compiled packages and other build artifacts.
- The default location of the module cache is \$GOPATH/pkg/mod. To use a different location, set the **GOMODCACHE** environment variable.

User-Defined Type System

- Go's type system provides two user defined types: interface, which is used for specifying contracts, and struct, which is used for creating concrete user-defined types
- Go has no classes
- Methods can be added to any type
- Go has no inheritance
- Types stand alone by themselves; they just are and have no hierarchy
- Methods aren't special; they're just functions

- A struct is a collection of named fields.
- Behaviours can be added to struct type by specifying method receivers.
- There are two of types of method receivers: value receiver and pointer receiver.
- Pointer receivers can be used for two scenarios: Mutates the state of types, and types that need more memory.
- Go's type system provides type embedding, which allows you to embedded struct types into other structs to create bigger types using composition.

- **Interfaces are behaviour contracts**
- Interface types provide contracts to concrete types, which lets you define behaviours for your objects.
- Interfaces are implicitly satisfied
- In Go, explicit declaration of interface implementation is not required. You just need to implement the methods defined in the interface into your struct type to implement an interface type.
- "In object-oriented programming, a protocol or interface is a common means for unrelated objects to communicate with each other" — Wikipedia

Interface: Design Guidelines

- Provides hidden implementations
- “The bigger the interface, the weaker the abstraction” — Rob Pike
- Return concrete types, receive interfaces as parameters
- “Be conservative in what you send, be liberal in what you accept from others” — Robustness Principle

- ***interface{}*** (empty interface) says nothing. The keyword ***any*** is an alias to `interface{}`
- ***struct{}*** (empty struct) is a struct type that has no fields
- Empty struct occupies zero bytes of storage

Struct Methods Vs Free Functions

- If package level variables are used by free functions, then refactor with:
 - Create structs by making package level variables as properties
 - Add methods to the struct
- Use structs for isolating dependencies by making dependencies as properties with interface types. Inject the dependencies from the place where you create instances of those struct types.
- Write free functions where you don't references any state and don't create dependent objects.
- In short, create structs for defining states and dependencies.

Concurrency

Concurrency VS Parallelism

“In programming, concurrency is the *composition* of independently executing processes, while parallelism is the simultaneous *execution* of (possibly related) computations. Concurrency is about *dealing with* lots of things at once. Parallelism is about *doing* lots of things at once.”

Rob Pike, Co-Designer, Go

Concurrency Primitives

- goroutines
- channels

Goroutines

- A goroutine is a function that is running concurrently
- Goroutines can be launched by placing the *go* keyword before a function
- Created and managed by Go runtime
- Executing based on a technique called ***m:n Scheduler*** (Maps *m* goroutines (goroutines are kind of **green threads**. A bit more specific word would be “**coroutines**”) to *n* OS threads)

Green threads are threads that are scheduled by a runtime library or virtual machine (VM) instead of natively by the underlying operating system.

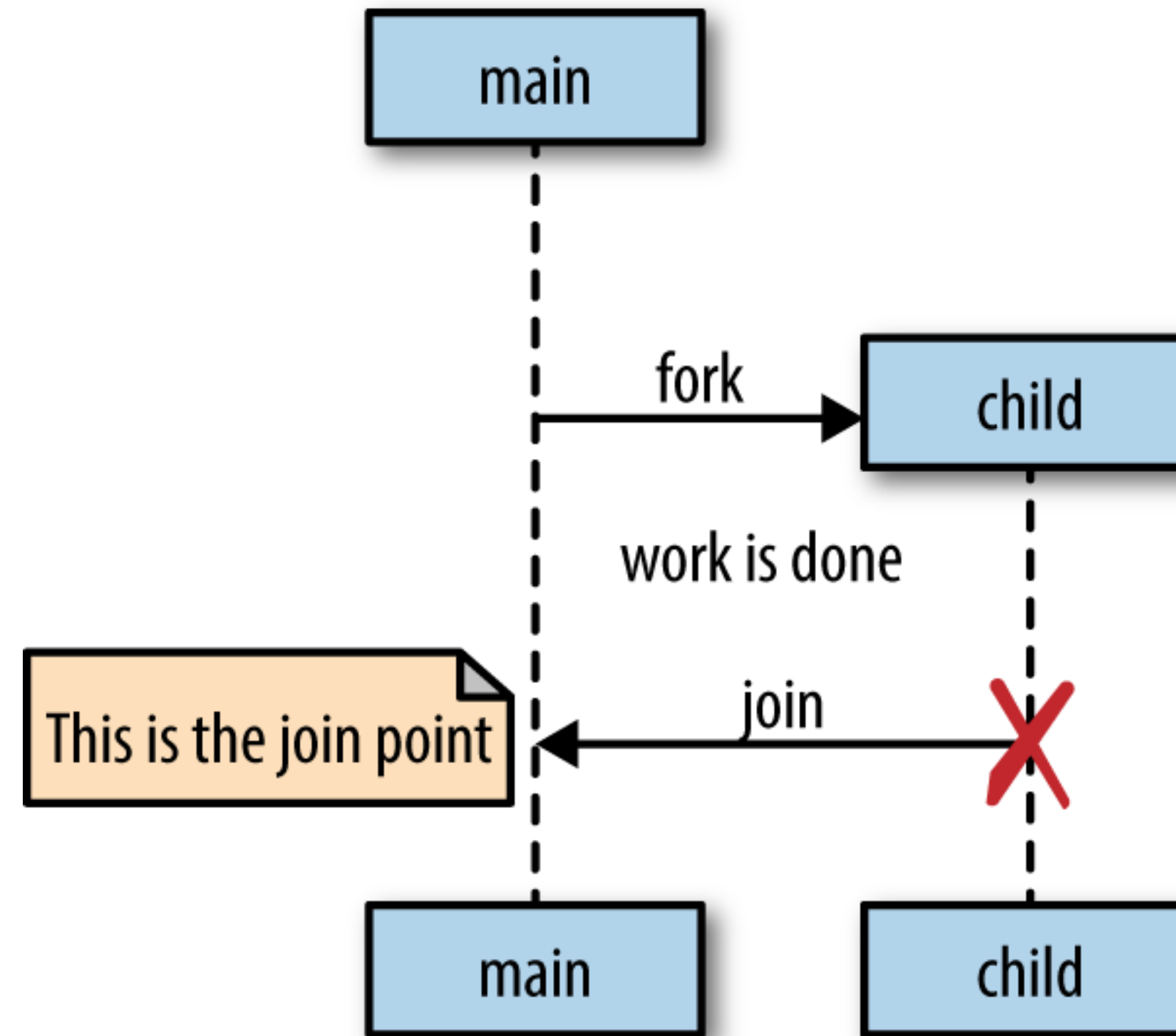
Go Scheduler

Go scheduler is a ***m:n scheduler***

Multiplexes (or schedules) ***m*** goroutines on ***n*** OS threads.

Scheduler is responsible to schedule the goroutines on limited number of OS threads

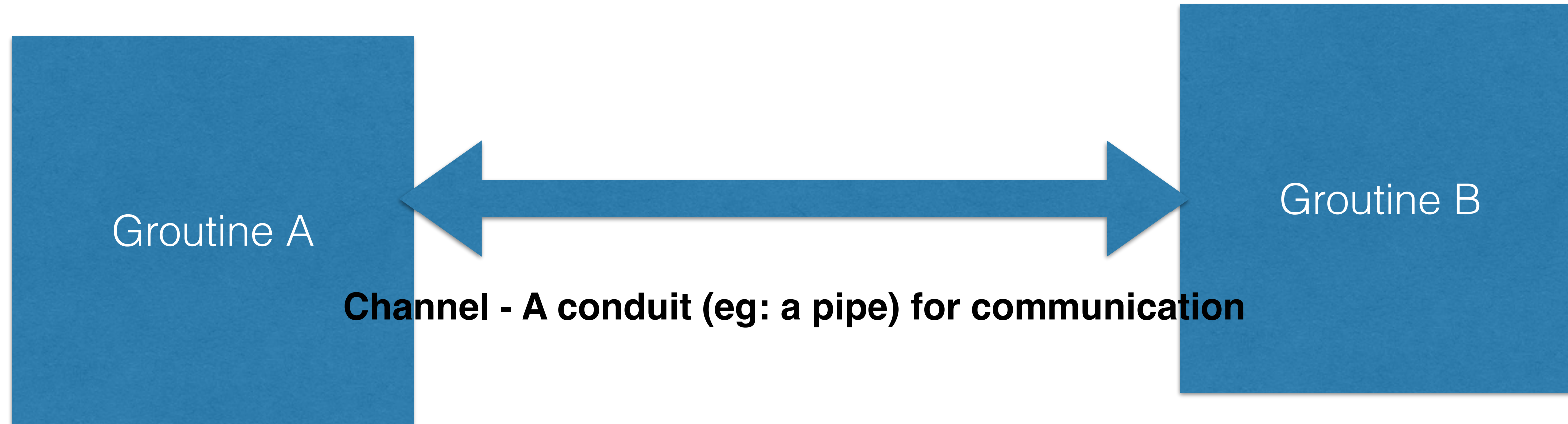
Fork-Join Model



```
func main() {  
    go foo()  
    go bar()  
    // continue doing other things  
}  
  
func foo() {  
    fmt.Println("foo")  
}  
  
func bar() {  
    fmt.Println("bar")  
}
```


Channels

Channels provide a simple mechanism for goroutines to communicate, and a powerful construct to build sophisticated concurrency patterns.



Channel Operations:

Send
Receive

Channels

- Channels are goroutine safe
- Channels carry messages between goroutines
- Channels cause blocking and unblocking

Types of Channel

- UnBuffered Channel
- Buffered Channel

Unbuffered Channel

- Synchronous communication
- Block sending goroutine or receiving goroutine until corresponding send or receive happening on the same channel

Buffered Channel

- Asynchronous communication
- Stores up to capacity elements and provides FIFO semantics

- Concurrency in Go is the ability for functions to run independent of each other.
- Go provides concurrency using two paradigms: goroutine and channel.
- A concurrently executing function in Go, is known as goroutine.
- Channel is the communication mechanism to send and receive data between goroutines.
- There are two types of channels are available: unbuffered channel and buffered channel.
- The built-in function `make` is used to initialize channels.

- Channels have two principal operations: Send, which writes message to channel, and Receive, which receives messages from channels.
- Once a channel is closed using built-in close function, no more values can be sent into the channel.
- The unbuffered channel is used to perform synchronous communication between goroutines and buffered channel is used for perform asynchronous communication.
- The unbuffered channel provides the exchange of data in a synchronous manner that ensures that a send operation on the channel from one goroutine, would be successfully delivered to another goroutine with a corresponding receive operation on the same channel.

- A buffered channel is capable of buffering values up to its capacity and it provides asynchronous communication for data exchange.
- A buffered channel is like a queue on which a send operation doesn't block any goroutine because of its capability for holding elements. The capacity of a buffered channel is determined when it is created by using make function.
- A receive operation on the closed channel receives default value of the channel's element type (eg: empty string for a channel of string)
- The Select expression is used make channel operations between multiple channels.

HTTP Programming

HTTP Programming Model

- A Request - Response Programming model
 - HTTP Client sends a request to the server
 - Server serves the request and sends back a response
- In Go, standard library package ***net/http***, provides the essential functionalities for writing HTTP applications.

Major components of net/http

- **ServeMux** (struct): HTTP request multiplexer (router)
- **Handler** (interface): Serves the HTTP requests. Writes headers and bodies into HTTP response

Handler interface

```
type Handler interface {
```

```
    ServeHTTP(ResponseWriter, *Request)
```

```
}
```

```
type ResponseWriter interface {
```

```
}
```

```
type Request struct {
```

```
}
```

HandlerFunc

HandlerFunc type is an adapter to allow the use of ordinary functions as HTTP handlers.

HandlerFunc is an implementation of Handler interface

```
type HandlerFunc func(ResponseWriter, *Request)
```

```
func (h HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) {  
}
```

```
func index(w ResponseWriter, r *Request) {}
```

```
// Create HandlerFunc value
```

```
handler := HandlerFunc (index)
```


HTTP Middleware

- Pluggable and self-contained piece of code that wraps web application handlers.
- Components that work as another layer in the request handling cycle, which can execute some logic before or after executing your HTTP application handlers.
- Great for implementing cross-cutting concerns: Authentication, authorization, caching, logging, etc.

```
func middlewareHandler(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        // Our middleware logic goes here before executing application handler  
        next.ServeHTTP(w, r)  
        // Our middleware logic goes here after executing application handler  
    })  
}
```


Generics

Generics

- Generics are a way of writing code that is independent of the specific types being used.
- With generics, you can declare and use functions or types that are written to work with any of a set of types provided by calling code.
- Functions and types are now permitted to have type parameters. A type parameter list looks like an ordinary parameter list, except that it uses square brackets instead of parentheses.

```
func SumNumbers[K comparable, V Number](m map[K]V) V {  
}
```

Package Context

- Package context defines the Context type, which carries deadlines, cancelation signals, and other request-scoped values across API boundaries and between processes.
- Context created using:
 - WithCancel
 - WithDeadline
 - WithTimeout
 - WithValue

```
func DoSomething(ctx context.Context, arg Arg) error {  
  
}
```

- Do not pass a nil Context, even if a function permits it. Pass context.TODO if you are unsure about which Context to use.
- Use context Values only for request-scoped data that transits processes and APIs, not for passing optional parameters to functions.
- The same Context may be passed to functions running in different goroutines; Contexts are safe for simultaneous use by multiple goroutines.

Thanks

Shiju Varghese

Senior Consulting Solutions Architect

<https://shijuvar.medium.com>

<https://github.com/shijuvar>

<https://linkedin.com/in/shijuvar>

<https://instagram.com/shijuvar>

Shiju Varghese's Go Masterclass: <https://bit.ly/shiju-go>